Analysis of Algorithm Project

Hannah Bregel, Grace Yanzito, and Lucy Dobson

September 2026

https://github.com/GraceYanzito/Analysis-of-Algorithms-Project

Problem: What is the longest loop at Silver Falls I can complete in T hours?

This is an algorithmic problem rather than a programming exercise because it cannot be solved using a loop or conditional statement. It requires finding the most consistent path within your data – the path will be defined as a path that fulfills the time and accessibility restraints defined by the participant. In an ideal instantiation of this problem, we would have a set number of disconnected trails, and the code would choose one (or more) based on the allotted time. The inputs we are using from many different interconnected paths so, in order for the computer to find the optimal path of the desired amount of time, the algorithm must evaluate different combinations of these paths first.

Finding the longest path is NP hard for a few important reasons:

- The number of possible solutions grows by a factorial rate, meaning you can't efficiently brute force it.
- A greedy algorithm will not work as well because the shortest (or longest in this case) path in not guaranteed, and it does not have reliable space-time complexity.
- There are no guaranteed efficient polynomial-time algorithm that can can work for all cases of our problem.

Because finding a longest cycle on a graph is an NP-hard problem in general, finding an exact solution is computationally difficult for large datasets. This high complexity justifies the use of the Held-Karp programming approach for our small graph points, and it is the reason why we compare with a heuristic to evaluate the trade-offs between computational efficiency and solution optimality.

The inputs for our problem include the names, distance (in miles), and time (in hours) requirements for each trail in Silver Falls National Park

- Park Map 1
- Park Map 2

The outputs for our problem include the distance (in miles) of the decided path, the amount of time (in hours) it will take for someone to complete this path, and the names of the trail-heads along the path.

Algorithms to Compare
Approach 1: Held-Karp
As Rosetta Code [1] states, the runtime using the algorithm would be $O(n^2 \cdot 2^n)$. This specific algorithm would be best for small inputs and produces a more precise output. One downfall of this algorithm is that the computational power is too great to use for data sets larger than 20 nodes.

Approach 2: Nearest Neighbor
As Alan Frieze and Wesley Pegden [2] state, the Nearest Neighbor heuristic works by choosing a vertex and building a sequence of paths where the next vertex minimizes the distance over all and returns the path it finds. This heuristic uses a Brute force method by starting at an arbitrary node, finding its closest neighbor, repeating until each node has been visited, and finally adding a edge connecting to the starting node. Because of this, this heuristic's output is different when different starting points are selected. Nearest Neighbor has a $O(n^2)$ runtime and is best for small inputs. Two downfalls for this algorithm are that it is not always precise and does not work best for large inputs. Even though the Nearest Neighbor sacrifices finding the absolute best path every time, it is more practical than requiring the large amount of computational power needed for Held-Karp.

Evaluation Plan We will be measuring the solution quality – defined as a consistent output that fulfills the specified requirements stated by the participant. We will run this program ten times per data size and then take the mean of these ten repetitions. In order to ensure our algorithm works with increasing inputs, we will create a generator for random trail networks with n junctions, using realistic distance and time weights. We will use at least on real part to ensure our algorithm has real-world application. We will use a plat graph with the x-axis being "Input Size" and the y-axis being "Operation Time" in seconds. Held-Karp will always find the optimal loop and its time doubles with every added junction; Nearest Neighbor runs in polynomial time and finds a loop within 15% - 20% of optimal on average but lacks a worst-case guarantee.

Plan and Roles

- 11/4: 1 algorithm running correctly on real inputs, output is consistent, and you have looked at your first plot. We will use the Held Karp algorithm first.
- 11/20: Both algorithms are working and begin making the presentation.
- 11/30: Code should be all working and the presentation should be finished and practiced.

Ownership:

- Lucy: Held-Carp algorithm
- Grace: Nearest Neighbor algorithm
- Hannah: Experiments and Plots implementations

One/two things that are most likely to go wrong and how will we avoid/fix them
One this that is mostly likely to go wrong is that Held-Karp becomes infeasible past about 20 nodes. The solution to this problem is to ensure when the $n \geq 20$ we will use our Nearest Neighbor algorithm instead.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, fourth edition, 2022.

[2] Alan Frieze and Wesley Pegden. The bright side of simple heuristics for the TSP. Mathematical Sciences, September 2024. Retrieved October 23, 2026.